

Configurable DNN Accelerator with Distributed SRAMs

Advising professor: Jin-Fu Li

Pei-yu Xie
Department of Electrical Engineering
National Central University
Taoyuan, the ROC
nanais198@g.ncu.edu.tw

Shu-yu Chou
Department of Electrical Engineering
National Central University
Taoyuan, the ROC
abc110501534@g.ncu.edu.tw

Abstract—In contemporary deep neural networks(DNNs) models, integrating depthwise-separable (DW) convolution is common for parameter reduction. However, traditional DNN accelerators often face low utilization rates during DW convolution computations. To address this, we propose an architecture featuring distributed buffers instead of a global one, enhancing efficiency. Moreover, our design allows for configurable optimization of utilization rates across different convolution types, ensuring peak performance. Additionally, we provide an analysis for determining the optimal configuration of processing elements (PEs) arrays based on factors like latency, operating frequency, and chip area, offering a refined approach to DNN acceleration.

Keywords—*accelerator, depthwise separable, configurable, distributed SRAMs*

I. INTRODUCTION

In the dynamic landscape of AI technology, the AI accelerator, rooted in the design principles of systolic arrays, plays a pivotal role in enhancing computational efficiency, particularly in training deep learning models.

As AI methodologies advance, convolutional neural networks (CNNs) have emerged as a cornerstone, offering various convolution types, shown in Fig. 1, each with distinct characteristics. Among these, Depthwise (DW) convolution has gained significant traction for its capacity to reduce parameter requirements in models, thus optimizing computational resources. As for the convolution with more than one channel, Fig. 2 illustrates their functioning methods.

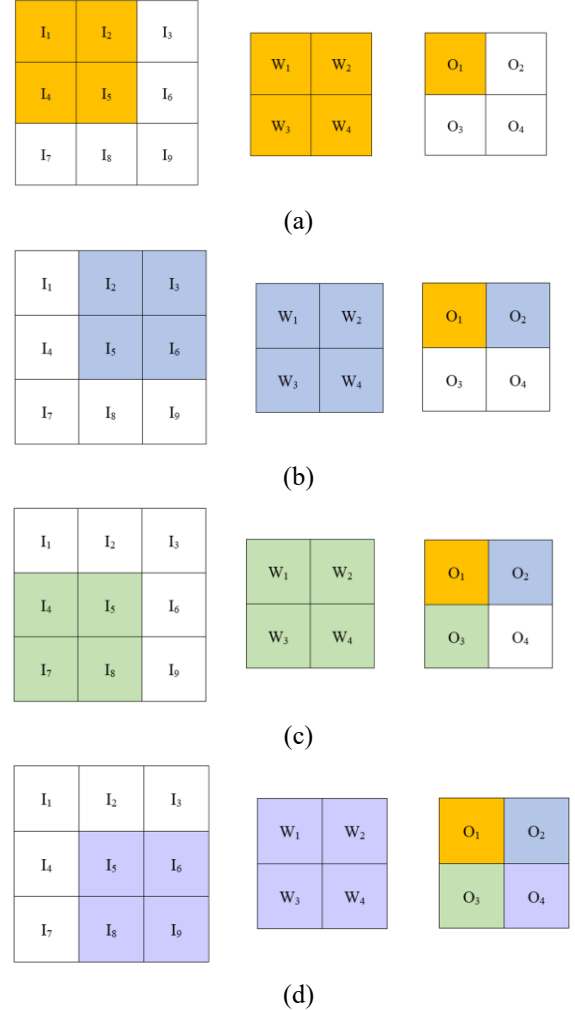
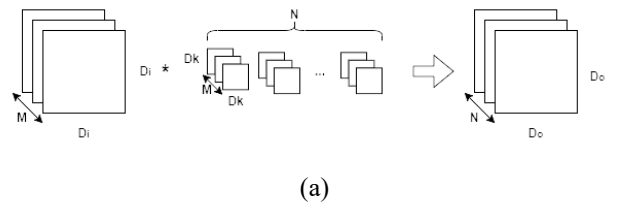


Fig. 1. Convolution calculation. For example, $O_1 = I_1 \times W_1 + I_2 \times W_2 + I_4 \times W_3 + I_5 \times W_4$



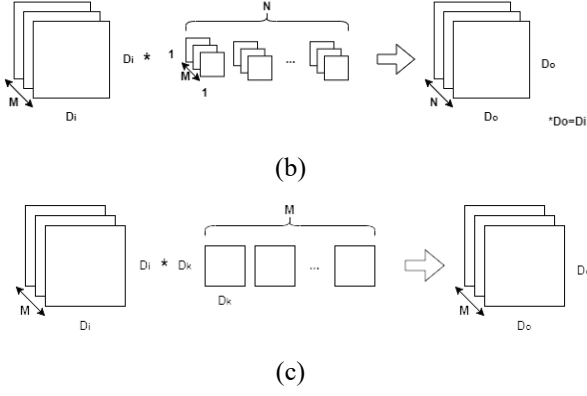


Fig. 2. Convolution types. (a) is the form of standard convolution and (b) is PW convolution ($D_k=1$). (c) is the form of DW convolution

However, traditional AI accelerators encounter challenges in fully harnessing the potential of DW convolutions due to their inherent lack of input reuse characteristics. This drawback is described with Fig. 3. To confront this limitation head-on, we propose a pioneering architecture for an AI accelerator, meticulously engineered with distributed Static Random-Access Memories (SRAMs). This novel design, showcased in Fig. 4, effectively addresses the efficiency gap encountered during DW convolution operations.

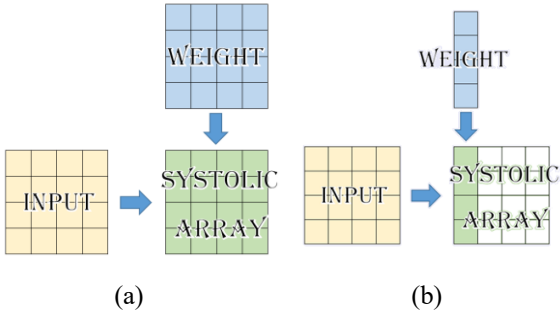


Fig. 3. (a) Standard/ PW and (b) DW PE utilization rate when using traditional accelerator

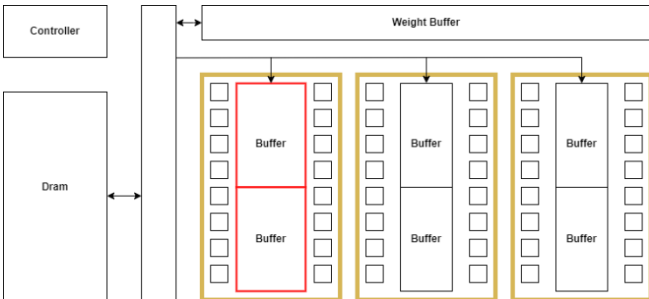


Fig. 4. Proposed architecture

In this project, we delve into the intricacies of our architecture, elucidating its underlying principles, design rationale, and performance benchmarks. Through rigorous experimentation

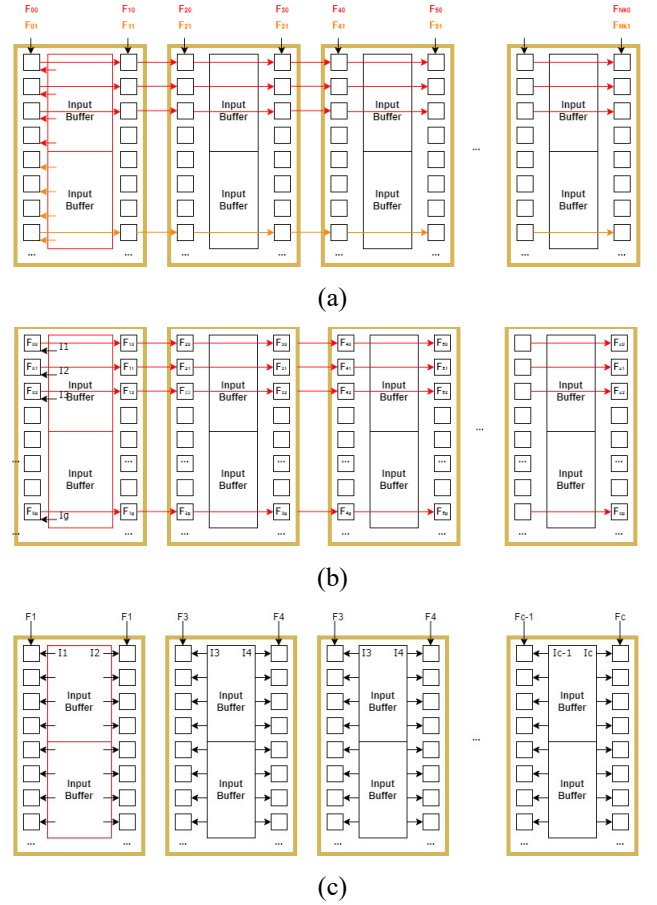
and analysis, we aim to demonstrate the efficacy of our proposed solution in maximizing hardware utilization and minimizing latency.

II. ARCHITECTURE OVERVIEW

Our proposed AI accelerator architecture is designed to meet the evolving demands of deep learning model training, leveraging a distributed arrangement of PEs and SRAMs to optimize computational efficiency.

Central to our architecture is the division into distinct units, each comprising an SRAM buffer flanked by two columns of PEs. This modular arrangement facilitates adaptive data processing tailored to different convolution types, ensuring optimal performance across diverse neural network architectures.

Fig. 5 illustrates the dynamic dataflow within our architecture across three prevalent convolution types. Notably, the initial buffer shoulders a heavier data load than subsequent buffers, reflecting the varying data requirements inherent to different convolution operations.



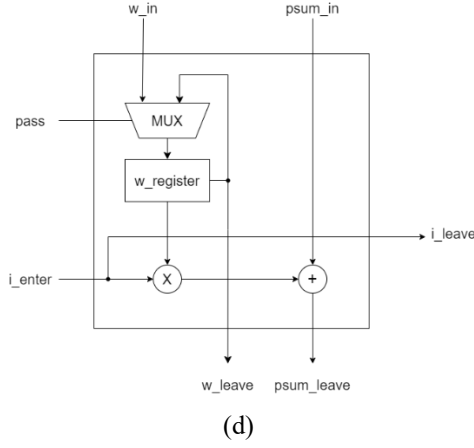


Fig. 5. Dataflow when performing (a) standard convolution, (b) PW convolution, and (c) DW convolution. (d) is the design of rach PE.

To empower practitioners with a comprehensive understanding of our architecture's capabilities, we offer a set of formulas for calculating the optimal size of the PE array, the tiling size of input feature maps, and the required minimum bandwidth of DRAM. The tiling strategy, shown in Fig. 6, minimizes the time to access DRAM. These formulas are indispensable tools for aligning the architecture with specified latency and area constraints, thereby enabling seamless integration into a wide array of computational environments.

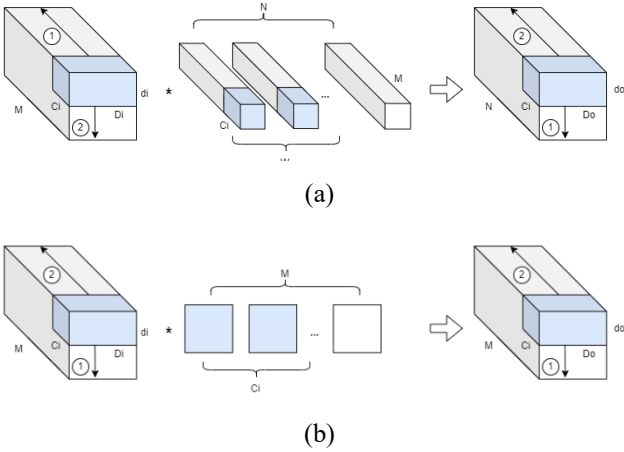


Fig. 6. Tiling strategy for (a) standard convolution and PW convolution and (b) DW convolution

III. METHODS

A. Hardware Design

Utilizing Verilog, we conduct comprehensive simulations to realize the hardware implementation of our architecture. The Verilog code encompasses the design of individual processing elements (PEs), the interconnections between each PE, and the input and output ports for the SRAMs. This hardware design process ensures the seamless integration and

efficient operation of our architecture within real-world computational environments.

B. Buffer Designs

We proceed to generate and optimize each buffer design. We craft buffer designs tailored to the unique requirements of our architecture. Subsequently, we employ Electronic Design Automation (EDA) tools to simulate the hardware implementation of each buffer design, ensuring optimal performance and compatibility within the broader architectural framework.

Through the synergistic integration of these methodological approaches, we strive to deliver a robust and efficient AI accelerator architecture capable of meeting the diverse computational demands of modern deep learning model training.

IV. IMPLEMENTATION RESULTS

A. Waveform

1) *Standard Convolution*: Add author names horizontally, moving to a third row if needed for more than 8 authors.



Fig. 7. Waveform of standard convolution

Partial verification: In “result[0][7]” in Fig. 6, we got:

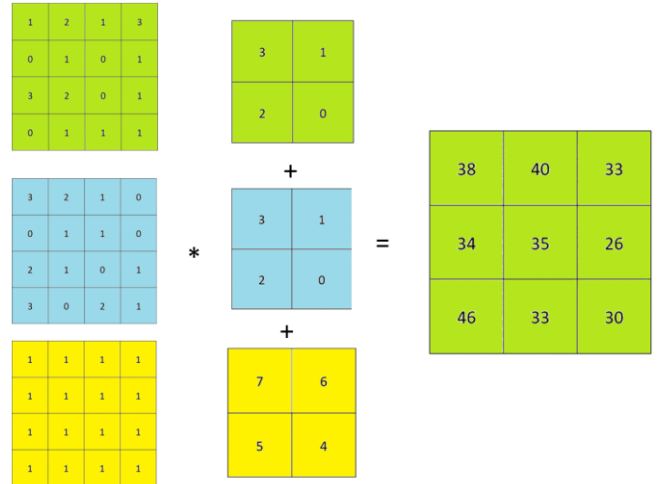


Fig. 8. Example of standard convolution

2) *Pointwise Convolution*: To change the default, adjust the template as follows.



Fig. 9. Waveform of PW convolution

Partial verification: In “result[2][2]” in Fig. 8, we got:

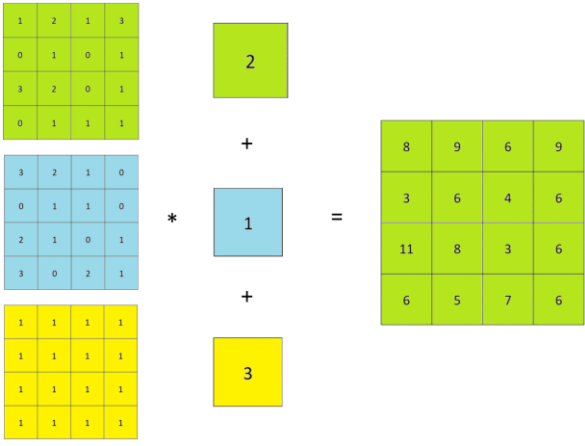


Fig. 10. Example of PW convolution

3) *Depthwise Convolution*: To change the default, adjust the template as follows.



Fig. 11. Waveform of DW convolution

Partial verification: In “result[0][3]” in Fig. 10., we got:

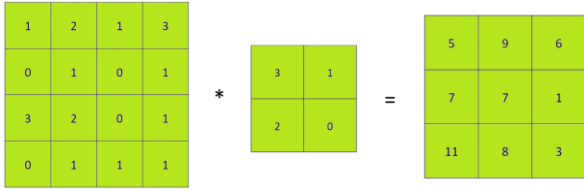


Fig. 12. Waveform of DW convolution

B. Area Report (CBDK_TSMC90GUTM_Arm_f1.0)

TABLE I. AREA WITH DIFFERENT ROWS

PE array (H*W)	Total Cell Area (μm ²)	Number of Pes
8*4	738066.842543	32
12*4	748530.185052	48
16*4	759035.157964	64

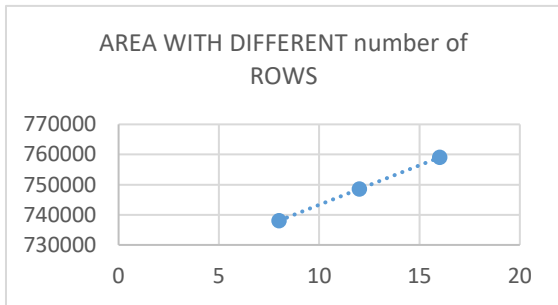


Fig. 13. Area comparison (rows)

TABLE II. AREA WITH DIFFERENT COLUMNS

PE array (H*W)	Total Cell Area (μm ²)	Number of Pes
8*4	738066.8425	32
8*6	1069511.889	48
8*8	1431149.092	64

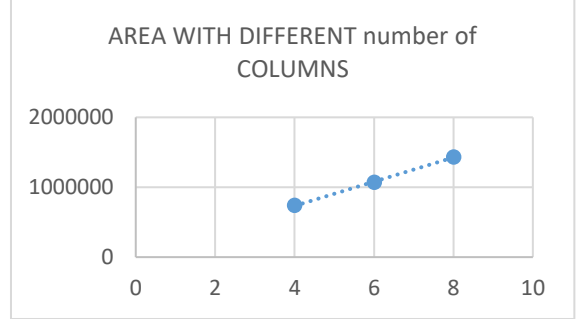


Fig. 14. Area comparison (columns)

Whether changing the column or row, the growth in area is linear. It can be observed that changing the column results in a larger area variation, which is due to the increased number of SRAM required.

C. Timing Report (CBDK_TSMC90GUTM_Arm_f1.0)

TABLE III. TIMING REPORT

Point	Incr	Path
clock clk' (rise edge)	1.50	1.50
Clock network delay (ideal)	0.00	1.50
u2/CLK (SRAM_B1_twin)	0.00	1.50 r
u2/Q[16] (SRAM_B1_twin)	0.92	2.42 r
U1209/Y (AOI222XL)	0.11	2.53 f
U1210/Y (INVXL)	0.06	2.60r
ifmp_reg[1][0]/D (DFFRQXL)	0.00	2.60r
clock clk (rise edge)	3.00	3.00
clock network delay (ideal)	0.00	3.00
ifmp_reg[1][0]/CK (DFFRQXL)	0.00	3.00r
library setup time	-0.15	2.85

Data Required Time (ns)	Slack (MET)
5.85	1.75

D. Utilization Rate Analysis

In the proposed architecture, the distinction between the traditional architecture and the new approach becomes evident, particularly when performing depthwise (DW) convolution. The key difference lies in the utilization of all columns, rather than just the one closest to the unique SRAM

buffer. This adjustment results in a utilization rate increase by a factor of W , where W represents the number of rows in the processing element (PE) array.

For the other two types of convolution, however, the utilization rates remain unchanged. This enhancement specifically benefits DW convolution by maximizing resource usage and efficiency, highlighting the advantage of the proposed architectural modification over traditional methods.

V. CONCLUSION

The PE model we've developed accurately computes Standard, Depthwise, and Pointwise operations. It can also perform secondary computations when the total number of kernels exceeds the number of PEs. In the case of Depthwise operations, our model significantly increases the utilization rate. Consequently, our architecture is particularly advantageous for training models that contain many Depthwise (DW) layers.

Based on these results, we recommend our architecture for models with a high number of DW layers. Looking ahead, our goal is to reduce the required area and complete the overall design, including the controller and DRAM access. This will enable us to further accelerate convolution speed for models with numerous DW layers, enhancing overall performance and efficiency.

ACKNOWLEDGMENT

We extend our sincere gratitude to Professor Jin-Fu Li, whose invaluable guidance and support were instrumental throughout this research. We also thank the graduate students and staff members of NCU for their assistance.

We appreciate the administrative staff, for their administrative support and assistance with logistical arrangements.

Our thanks also go to the IEEE website for providing access to valuable resources and relevant literature.

We are grateful to all who contributed to this work.

REFERENCES

- [1] V. Sze, Y. -H. Chen, T. -J. Yang and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," in *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295-2329, Dec. 2017, doi: 10.1109/JPROC.2017.2761740.
- [2] Y. -H. Chen, T. Krishna, J. Emer and V. Sze, "14.5 Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," 2016 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2016, pp. 262-263, doi: 10.1109/ISSCC.2016.7418007.
- [3] D. Yang and Z. Luo, "A Parallel Processing CNN Accelerator on Embedded Devices Based on Optimized MobileNet," in *IEEE Internet of Things Journal*, vol. 10, no. 21, pp. 18844-18852, 1 Nov. 2023, doi: 10.1109/JIOT.2023.3277869.
- [4] R. Xu, S. Ma, Y. Wang, Y. Guo, D. Li and Y. Qiao, "Heterogeneous Systolic Array Architecture for Compact CNNs Hardware Accelerators," in *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 2860-2871, 1 Nov. 2022, doi: 10.1109/TPDS.2021.3129647.
- [5] J. Yang, H. Zheng and A. Louri, "Versa-DNN: A Versatile Architecture Enabling High-Performance and Energy-Efficient Multi-DNN Acceleration," in *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 2, pp. 349-361, Feb. 2024, doi: 10.1109/TPDS.2023.3340953.
- [6] H. Li, L. Gong, C. Wang and X. Zhou, "A flexible dataflow CNN accelerator on FPGA," 2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW), Bangalore, India, 2023, pp. 302-304, doi: 10.1109/CCGridW59191.2023.00065.
- [7] R. Xu, S. Ma, Y. Wang and Y. Guo, "CMSA: Configurable Multi-directional Systolic Array for Convolutional Neural Networks," 2020 IEEE 38th International Conference on Computer Design (ICCD), Hartford, CT, USA, 2020, pp. 494-497, doi: 10.1109/ICCD50377.2020.00089.
- [8] J. Ou, X. Li, Y. Sun and Y. Shi, "A Configurable Hardware Accelerator Based on Hybrid Dataflow for Depthwise Separable Convolution," 2022 4th International Conference on Advances in Computer Technology, Information Science and Communications (CTISC), Suzhou, China, 2022, pp. 1-5, doi: 10.1109/CTISC54888.2022.9849816.